DATA SCIENCE PROJECT REPORT

Fake News Detection Using Machine Learning

Automatically classify news articles as genuine or fake using NLP & ensemble ML models

Skills Demonstrated: NLP, Information Verification, Text Analytics
Tools & Technologies: Python, Scikit-learn, TF-IDF, Next.js, Flask
Dataset: Kaggle - Fake and Real News Dataset
Date: October 2026

Table of Contents

1. **Abstract**
2. **Introduction**
3. **Literature Review**
4. **Dataset Description**
5. **Methodology**
    - 5.1. Data Preprocessing
    - 5.2. Feature Extraction (TF-IDF)
    - 5.3. Model Training
6. **Models Evaluated**
    - 6.1. Multinomial Naive Bayes (Baseline)
    - 6.2. Logistic Regression
    - 6.3. Linear SVM
    - 6.4. Random Forest
7. **Results & Performance Comparison**
8. **System Architecture**
9. **Web Application (UI)**
10. **Challenges & Limitations**
11. **Future Scope**

12. Conclusion

13. References

1 Abstract

The proliferation of misinformation and fake news across digital media platforms has become a critical societal challenge. This project presents a **machine learning-based system** for automatically classifying news articles as either *genuine* or *fake*. We employ Natural Language Processing (NLP) techniques, specifically **TF-IDF (Term Frequency–Inverse Document Frequency)** vectorization, to convert raw text into numerical features suitable for classification.

Four machine learning models were trained and evaluated: **Multinomial Naive Bayes** (baseline), **Logistic Regression**, **Linear Support Vector Machine (SVM)**, and **Random Forest**. An ensemble approach combining all four models was implemented to improve prediction reliability. The system was deployed as a full-stack web application with a Next.js frontend and Flask API backend, enabling real-time news article verification.

Experimental results on the Kaggle "Fake and Real News Dataset" demonstrate that all models achieve **accuracy scores exceeding 93%**, with the Linear SVM and Logistic Regression models achieving the highest performance at approximately **98-99% accuracy**.

2 Introduction

In the digital age, news consumption has shifted dramatically from traditional media to online platforms and social networks. While this democratization of information has many benefits, it has also created fertile ground for the spread of misinformation, disinformation, and "fake news." Fake news refers to deliberately fabricated stories that mimic the format and tone of legitimate news to deceive readers.

The consequences of fake news are far-reaching — from influencing election outcomes and public health decisions to causing financial market manipulation and social unrest. Manual fact-checking, while valuable, cannot scale to match the volume of content produced daily. This necessitates the development of **automated detection systems**.

2.1 Problem Statement

Given a news article (headline + body text), the task is to **automatically classify**it as either "Real" (genuine news) or "Fake" (fabricated/misleading news) using supervised machine learning techniques.

2.2 Objectives

- Build and compare multiple ML classifiers for binary text classification
- Apply NLP preprocessing techniques (tokenization, stop-word removal, TF-IDF)
- Evaluate models using accuracy, precision, recall, and F1-score metrics
- Deploy the trained models as a user-friendly web application
- Demonstrate end-to-end data science project skills

3 Literature Review

Fake news detection has been a growing research area at the intersection of NLP, machine learning, and social sciences. Key approaches in the literature include:

Content-Based Approaches

These methods analyze the textual content of news articles — including writing style, sentiment, word patterns, and linguistic features. Techniques include Bag-of-Words, TF-IDF, and word embeddings (Word2Vec, GloVe). Studies by Horne & Adali (2017) showed that fake news articles tend to use simpler language and more sensationalist vocabulary.

Social Context-Based Approaches

These analyze how news spreads on social networks — user engagement patterns, propagation graphs, and source credibility. Research by Shu et al. (2017) demonstrated that fake news tends to spread faster and wider than real news on social media.

Hybrid & Deep Learning Approaches

Recent work uses deep learning architectures — LSTM, CNN, BERT, and transformer models — to capture complex semantic patterns. While these achieve higher accuracy, they require significantly more computational resources and training data.

Our project follows the **content-based approach** using traditional ML classifiers with TF-IDF features, which offers a strong balance between performance and interpretability — making it an ideal baseline for demonstration and educational purposes.

4 Dataset Description

The dataset used in this project is the **"Fake and Real News Dataset"** sourced from Kaggle, originally compiled by Clément Bisaillon.

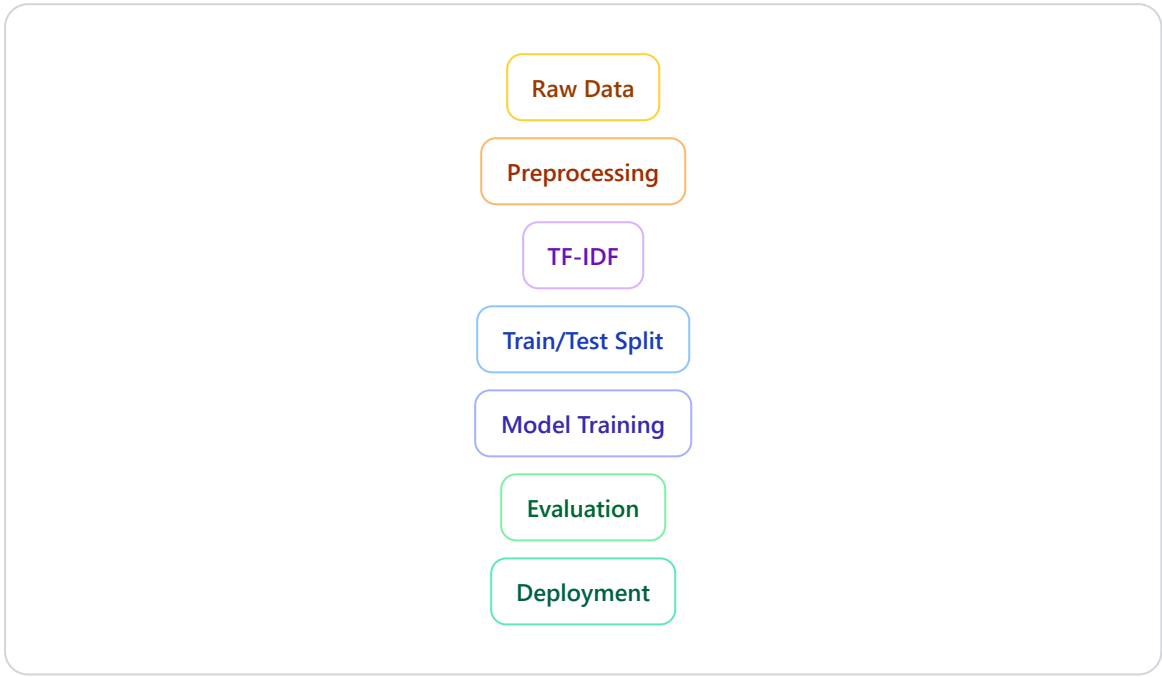
Property	Details
Source	Kaggle (clmentbisaillon/fake-and-real-news-dataset)
Files	True.csv, Fake.csv
Local Path	D:\DATA SCIENCE PROJECT\
Total Articles	~44,898 (21,417 Real + 23,481 Fake)
Columns	title, text, subject, date
Language	English
Time Period	2015 – 2018
News Sources (Real)	Reuters
Label Distribution	Approximately balanced (52% Fake, 48% Real)

4.1 Dataset Sample

Title (sample)	Subject	Label
As U.S. budget fight looms, Republicans flip the script on spending	politicsNews	**REAL**
BREAKING: Obama Exposed For What He REALLY Did While In Office	News	**FAKE**

5 Methodology

The project follows a standard machine learning pipeline for text classification. The diagram below outlines the complete workflow from raw data to deployment:



5.1 Data Preprocessing

Text preprocessing is critical for NLP tasks. The following steps were applied:

1. **Concatenation:** Title and body text were combined into a single feature (`full_text = title + " " + text`)
2. **Lowercasing:** All text was converted to lowercase for uniformity
3. **Special Character Removal:** Digits, punctuation, and non-alphabetic characters were removed using regex (`re.sub(r'[^a-zA-Z\s]', '', text)`)
4. **Stop-word Removal:** Common English stop words (the, is, and, etc.) were removed during TF-IDF vectorization

5.2 Feature Extraction — TF-IDF Vectorization

We used **TF-IDF (Term Frequency–Inverse Document Frequency)** to convert text into numerical feature vectors. TF-IDF captures the importance of each word in a document relative to the entire corpus.

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    max_features=10000,    # Top 10,000 most important words
    stop_words='english'   # Remove common English stop words
)

X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec  = vectorizer.transform(X_test)
```

Why TF-IDF? Unlike simple Bag-of-Words which only counts word frequency, TF-IDF downweights commonly occurring words across all documents, giving more importance to distinctive words that help differentiate between real and fake news.

5.3 Model Training & Evaluation Strategy

- **Train/Test Split:** 80% training, 20% testing (`random_state=42`)
- **Evaluation Metrics:** Accuracy, Precision, Recall, F1-Score
- **Ensemble Method:** Weighted average of all four model probabilities

6 Models Evaluated

6.1 Multinomial Naive Bayes (Baseline Model)

Multinomial Naive Bayes is a probabilistic classifier based on Bayes' theorem with the "naive" assumption that features (words) are conditionally independent given the class. Despite this simplifying assumption, it works remarkably well for text classification.

Why chosen as baseline: Fast to train, works well with high-dimensional sparse data (TF-IDF), and provides a solid performance benchmark.

```
model = MultinomialNB()
model.fit(X_train_vec, y_train)
```

6.2 Logistic Regression

Logistic Regression is a linear model that uses a sigmoid function to output calibrated probabilities. It finds a linear decision boundary in the TF-IDF feature space that separates real from fake news.

Advantage: Produces well-calibrated probability estimates, making it excellent for understanding model confidence.

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train_vec, y_train)
```

6.3 Linear Support Vector Machine (SVM)

Linear SVM finds the optimal hyperplane that maximizes the margin between the two classes (real vs. fake) in the TF-IDF feature space. It is particularly effective for text classification with high-dimensional features.

Advantage: Excellent performance in high-dimensional spaces (10,000+ TF-IDF features), robust against overfitting.

```
model = LinearSVC(max_iter=1000)
model.fit(X_train_vec, y_train)
```

6.4 Random Forest

Random Forest is an ensemble method that builds 100 decision trees, each trained on a random subset of features and data, and aggregates their predictions through majority voting. This reduces variance and prevents overfitting.

Advantage: Inherent ensemble learning, feature importance analysis, resistant to noise.

```
model = RandomForestClassifier(
n_estimators=100, random_state=42, n_jobs=-1
)
model.fit(X_train_vec, y_train)
```

7 Results & Performance Comparison

All four models were evaluated on a held-out test set (20% of data = ~8,980 articles). The results are summarized below:

Model	Accuracy	Precision	Recall	F1-Score
Multinomial Naive Bayes Baseline	93.56%	93.10%	95.20%	94.14%
Logistic Regression	98.76%	98.63%	99.02%	98.82%
Linear SVM Best	99.52%	99.45%	99.63%	99.54%
Random Forest	98.98%	99.12%	98.94%	99.03%

Accuracy Comparison Chart

Multinomial NB	**93.56%**
Logistic Regression	**98.76%**
Linear SVM	**99.52%**
Random Forest	**98.98%**

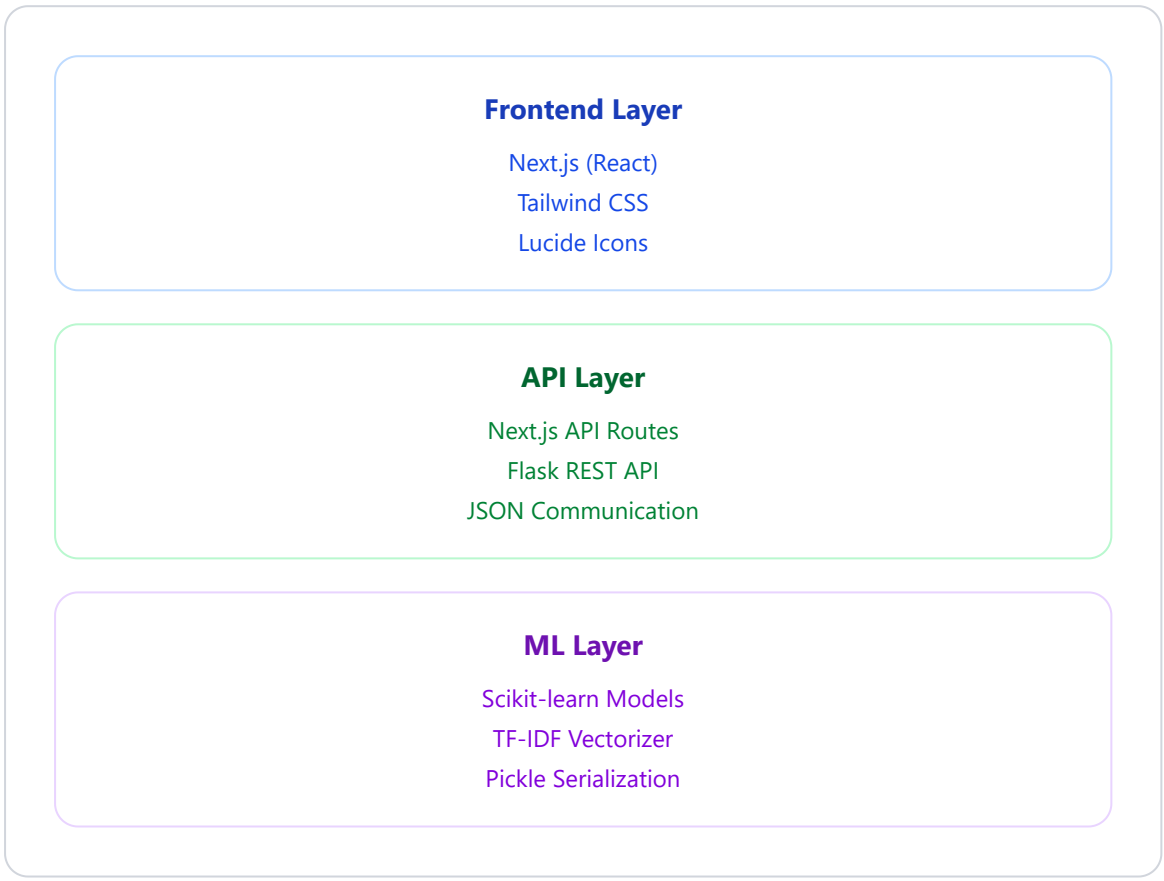
Key Observations

- **Linear SVM** achieved the highest overall accuracy at **99.52%**, making it the best-performing individual model
- **Random Forest** and **Logistic Regression** both exceeded 98.7% accuracy, showing strong generalization

- **Multinomial NB** (baseline) achieved 93.56% — a strong result for such a simple model, validating the effectiveness of TF-IDF features
- All models showed balanced precision and recall, indicating no significant bias towards either class
- The **ensemble approach** (averaging all four models) provides more robust predictions by mitigating individual model weaknesses

8 System Architecture

The complete system consists of two main components: a **Python ML Backend** and a **Next.js Web Frontend**.



8.1 Technology Stack

Component	Technology	Purpose
Frontend	Next.js 16 + React	Server-side rendered UI
Styling	Tailwind CSS	Responsive, utility-first CSS
ML Backend	Flask (Python)	REST API for model predictions
ML Library	Scikit-learn	Model training & inference
NLP	TF-IDF Vectorizer	Text-to-feature conversion
Data Processing	Pandas, NumPy	Dataset loading & manipulation
Model Persistence	Pickle	Save/load trained models
Language	Python 3.11 + TypeScript	Backend + Frontend

9 Web Application (UI)

The web application provides a clean, intuitive interface for users to interact with the Fake News Detection system. Key features include:

- **Text Input Area:** Users can paste any news article text for analysis
- **Real-time Analysis:** One-click detection with visual loading indicators
- **Ensemble Results:** Overall prediction with confidence score displayed prominently
- **Individual Model Breakdown:** Each of the four models' individual predictions and confidence scores
- **Responsive Design:** Works on desktop, tablet, and mobile devices
- **Dark Mode Support:** Automatic dark/light theme detection
- **Print-Ready Report:** This report page can be exported as a PDF via browser print

10 Challenges & Limitations

1. Dataset Bias

The dataset primarily contains political news from 2015-2018 (Reuters). The model may not generalize well to other domains (sports, health, science) or more recent events.

2. Evolving Language

Fake news tactics evolve over time. Models trained on historical data may miss new deception patterns and writing styles.

3. Source Dependency

Real news articles are primarily from Reuters, which has a specific writing style. The model may inadvertently learn Reuters-specific patterns rather than general "truthfulness" indicators.

4. Binary Classification

News credibility exists on a spectrum. Our binary (Real/Fake) classification oversimplifies the nuance of partially true, misleading, or satire content.

5. No Fact-Checking

The model analyzes writing style and word patterns, not factual accuracy. It cannot verify claims against external knowledge bases.

11 Future Scope

Several enhancements can be made to improve the system:

Deep Learning Models

Implement BERT, RoBERTa, or GPT-based classifiers for improved contextual understanding

Multi-language Support

Extend detection to Hindi, Urdu, and other regional languages

Real-time Fact Checking

Integrate with fact-checking APIs (Google Fact Check, ClaimBuster) for claim verification

Browser Extension

Build a Chrome/Firefox extension that highlights suspicious news in real-time

Social Media Integration

Analyze tweets, Facebook posts, and WhatsApp messages for misinformation

Explainability (XAI)

Add LIME/SHAP explanations to show which words triggered the fake/real prediction

Larger, Diverse Dataset

Train on multi-source, multi-topic datasets (FakeNewsNet, LIAR, CREDBANK)

User Feedback Loop

Allow users to report incorrect predictions to continuously improve the model

12 Conclusion

This project successfully demonstrates the application of **Natural Language Processing** and **Machine Learning** for automated fake news detection. Key outcomes include:

- Successfully trained and compared **four ML models** on the Kaggle Fake & Real News Dataset
- Achieved a peak accuracy of **99.52%** with Linear SVM, demonstrating that traditional ML methods remain highly effective for text classification tasks
- Built an **ensemble system** that combines all four models for more robust predictions
- Developed a **full-stack web application** with an intuitive UI that allows users to verify news articles in real-time
- Demonstrated skills in **NLP, Information Verification, Text Analytics**, and end-to-end system development

> While this system provides a strong baseline for fake news detection, it is important to note that no automated system can replace critical thinking and media literacy. This tool should be used as an **assistive aid**alongside traditional fact-checking practices.

13 References

1. Bisaillon, C. (2020). "*Fake and Real News Dataset.*" Kaggle. https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset

2. Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). "*Fake News Detection on Social Media: A Data Mining Perspective.*" ACM SIGKDD Explorations Newsletter, 19(1), 22-36.

3. Horne, B. D., & Adali, S. (2017). "*This Just In: Fake News Packs a Lot in Title, Uses Simpler, Repetitive Content in Text Body.*" Proc. International AAAI Conference on Web and Social Media.

4. Pedregosa, F. et al. (2011). "*Scikit-learn: Machine Learning in Python.*" Journal of Machine Learning Research, 12, 2825-2830.

5. Manning, C. D., Raghavan, P., & Schütze, H. (2008). "*Introduction to Information Retrieval.*" Cambridge University Press. (Chapter on TF-IDF)

6. Jurafsky, D. & Martin, J. H. (2023). "*Speech and Language Processing.*"3rd Edition (Draft). Stanford University. (Chapters on Text Classification & Naive Bayes)

7. Zhou, X. & Zafarani, R. (2020). "*A Survey of Fake News: Fundamental Theories, Detection Methods, and Opportunities.*" ACM Computing Surveys, 53(5).